

Kernel Recipes

# Linux Virtualization Based Security (LVBS): What, Why, and How

Thara Gopinath, Linux Systems Group, Microsoft

# Linux Virtualization Based Security

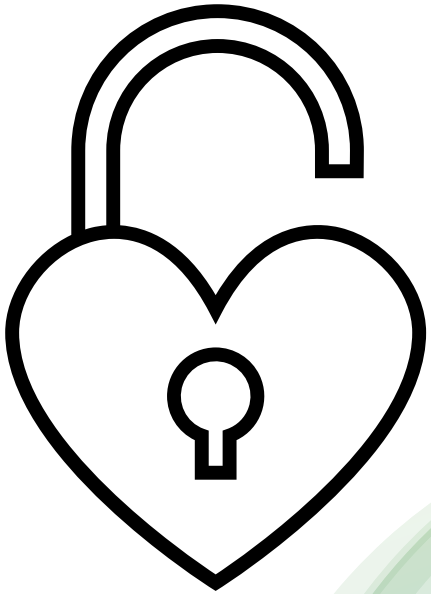
- Use virtualization to provide enhanced security for the guest/host OS, by establishing a higher privileged environment within the guest/host
  - leveraging the hypervisor security boundary
- Protect the integrity of security-critical operating system structures.
- Prevent bypass of OS security mechanisms and policies.
- Support a Trusted Execution Environment (TEE) for running security applications
- **Layered Security Approach :** Even if kernel compromise occurs, hypervisor enforces critical protections to maintain system security.

# Motivation

- Linux security is trailing proprietary solutions across Linux and other OSs.
- Linux Kernel vulnerabilities have been steadily rising and getting exploited in the wild. User space can exploit vulnerabilities like zero-day exploits and CVEs.
  - https://www.usenix.org/conference/usenixsecurity24/presentation/maar-slubstick
- Our goal is to:
  - Harden the kernel by enforcing protections, which cannot be turned off by a malicious kernel
  - Ensure that critical system assets (keys, critical kernel data structures) are inaccessible and/or untampered, even if the kernel gets compromised

# Kernel Hardening

- **Enforce critical kernel resource protection from outside the guest/host kernel in a higher privileged environment within the VM/host**
  - Lock down system critical register
  - Enforce W^X permissions on kernel memory space
  - Enforce read-only permissions on critical data structures



- **Maintain kernel integrity through the higher privileged environment**
  - allow only authenticated modules
  - Block attempts to disable set protections

- **Defense in Depth**

# Secure Enclaves

- **Trusted Execution Environment**
  - Enable trusted applications to run isolated within virtual machines at a higher privilege level than guest/host kernels.

- **Use Cases for Secure Enclaves**
  - Secure logging
  - Secure key management
  - Credential management
  - Virtual hardware security module



# Threat Model

- Security Goal: Protect kernel from a user space attacker exploiting a kernel vulnerability.

- Kernel is benign, but vulnerable (i.e. unpatched CVE or 0day)
  - Secure Boot verifies authenticity & integrity

- Guest kernel is trusted until the first unverified user space process starts running (i.e. init)
  - Prior to that, kernel is verified by Secure Boot. We assume that the kernel, even though it can be vulnerable, has not been compromised yet.

- After that, we assume that the kernel is compromised. For example, an unverified user application has exploited a kernel vulnerability, which granted kernel privileges to the attacker.

**Trusted Computing Base Components**

- Hypervisor, host OS, and hardware form the Trusted Computing Base that supports kernel security enforcement.

# Open-Source, Hypervisor-Agnostic Architecture

**Hypervisor and/or Platform Agnostic Integration**

**Abstract out hypervisor and platform details enabling integration with any virtual privilege level concepts out there.**

**Hyper-V VTL**

**AMD VMPL**

**Intel TD Partition**

**Arm CCA Plane**



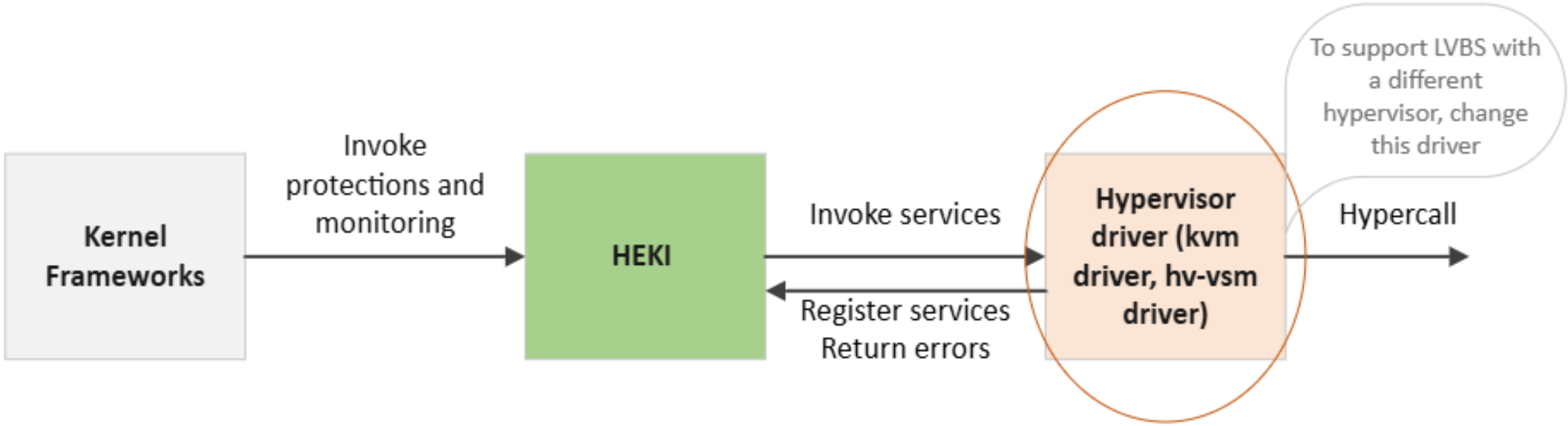
# Hardware Requirements

- **Second Level Address Translation**
  - EPT (Intel), RVI (AMD), Stage 2 Page Tables (Arm)
  - Secondary complementary layer of memory permissions only controlled by the higher privileged environment

- CPU features that allow to differentiate between kernel and user execute mode for memory (**Mode Based Execution Control**)

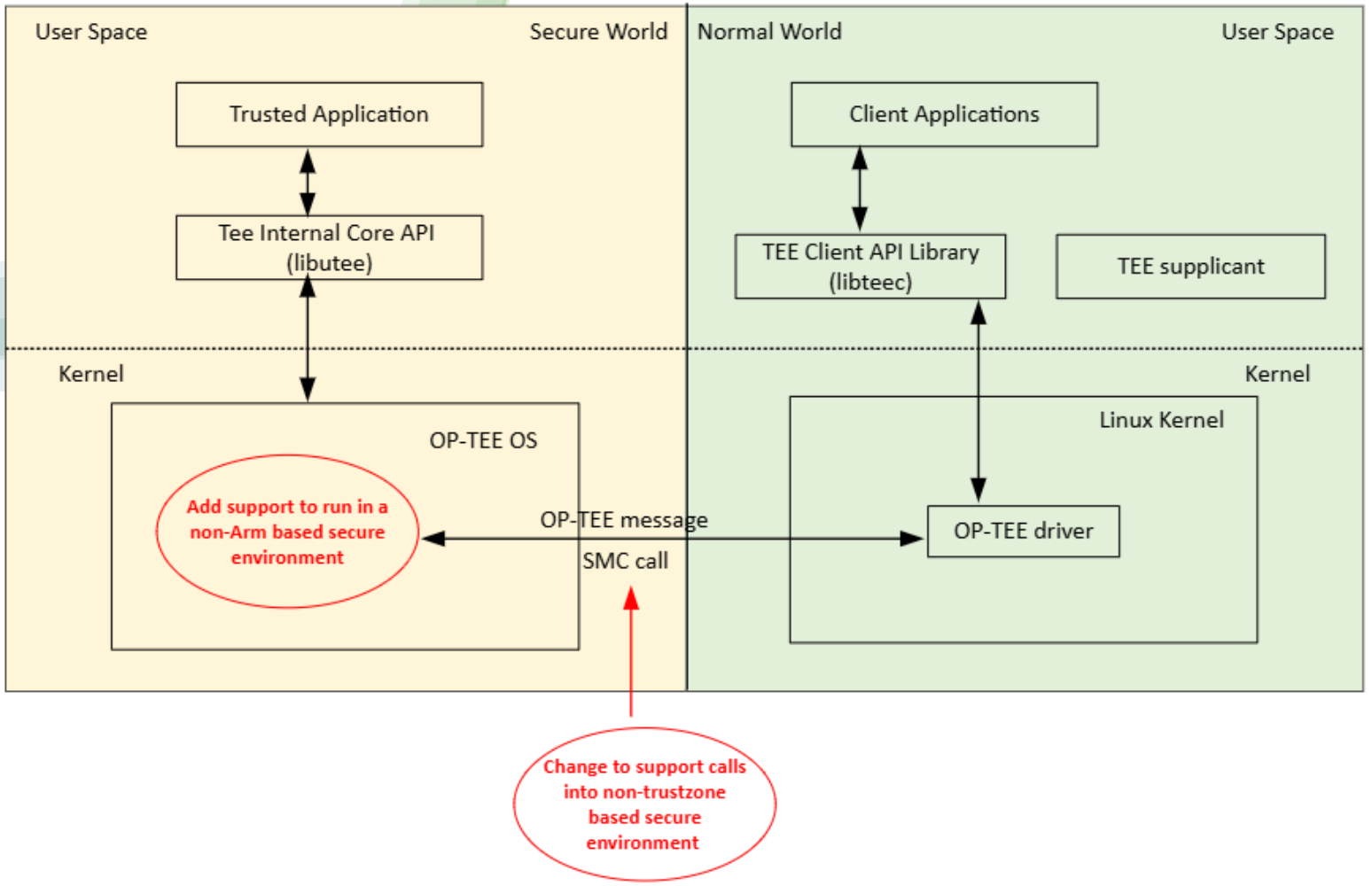
# Hypervisor Enforced Kernel Integrity (HEKI)

- Agnostic abstraction layer for Kernel Integrity



# HEKI APIs

| API Name | When Called | Purpose |
|---|---|---|
| **heki_late_init** | Late boot | Lock critical registers, set initial kernel memory protections, mark transition to authenticated modifications |
| **heki_validate_module** | During module loading | Authenticate and verify module memory before granting execute/read-only permissions |
| **heki_free_module_init** | After module initialization | Relax permissions for init code/data so pages can be reused by kernel |
| **heki_unload_module** | During module unload | Free module memory; policy consideration needed for LVBS module unload |
| **heki_text_poke** | Text patching (static calls, jump labels) | Ensure text patches are made from registered locations and with validated code; requires community alignment |
| **heki_copy_secondary_key** | Key management | Securely copy secondary keys |
| **heki_store_blacklist_raw_hashes** | Blacklist management | Store blacklisted hashes |

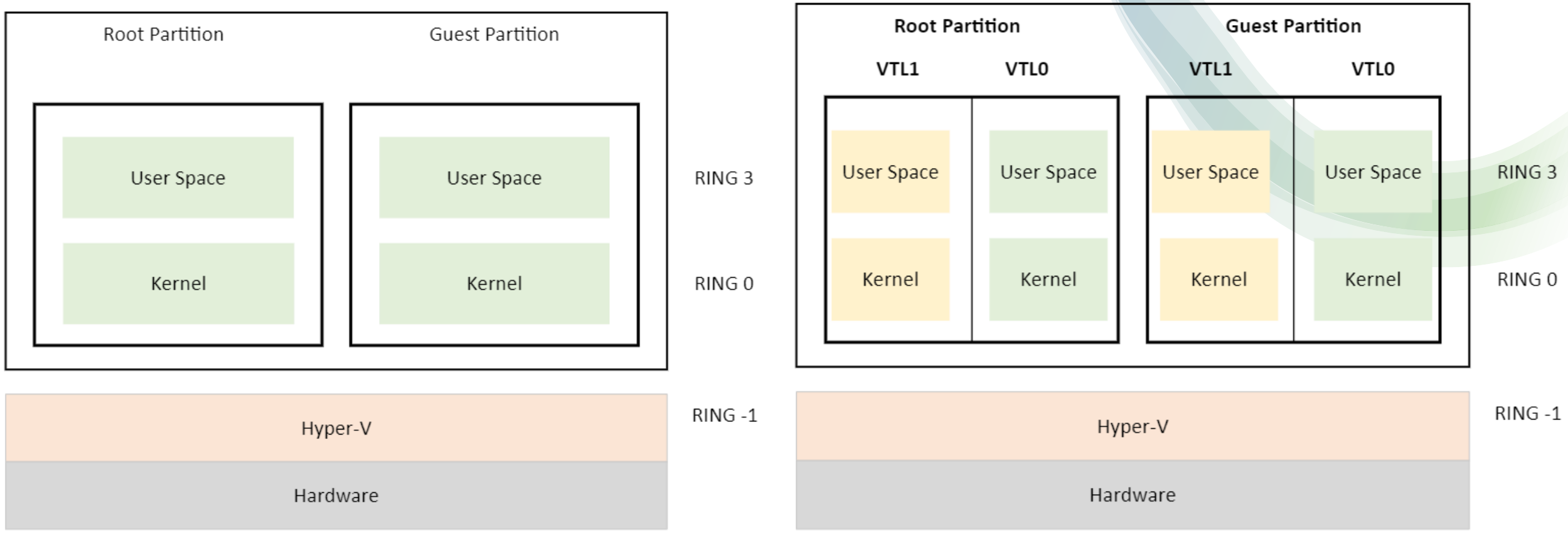



# Optee (Secure Enclaves)

- Support for Client and Internal TEE APIS adhering to GlobalPlatform API Specification
- Well defined and documented interfaces
- Active developer community and actively maintained

# Hyper-V Architecture

# Virtual Secure Mode and VTLs

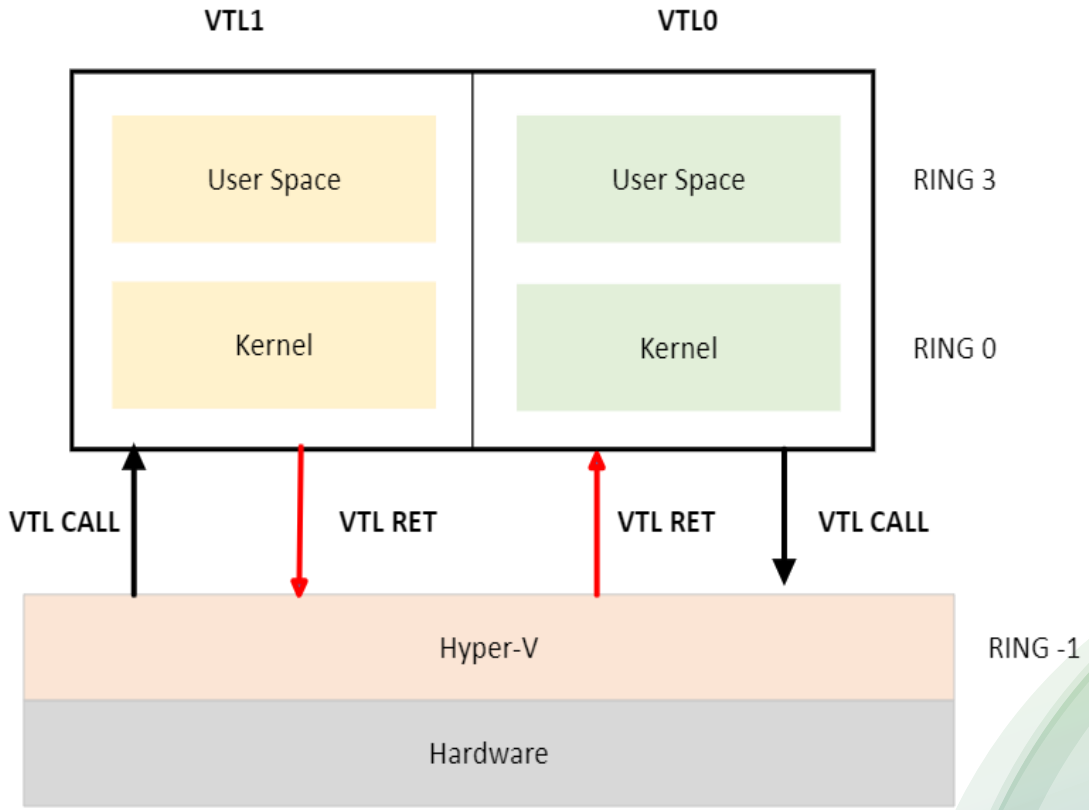



https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-vbs

# Control Interface

- Synchronous : Explicit VTL Call and Return
- Asynchronous : Interrupt based entry and explicit return

- Higher VTL gets precedence over lower VTL



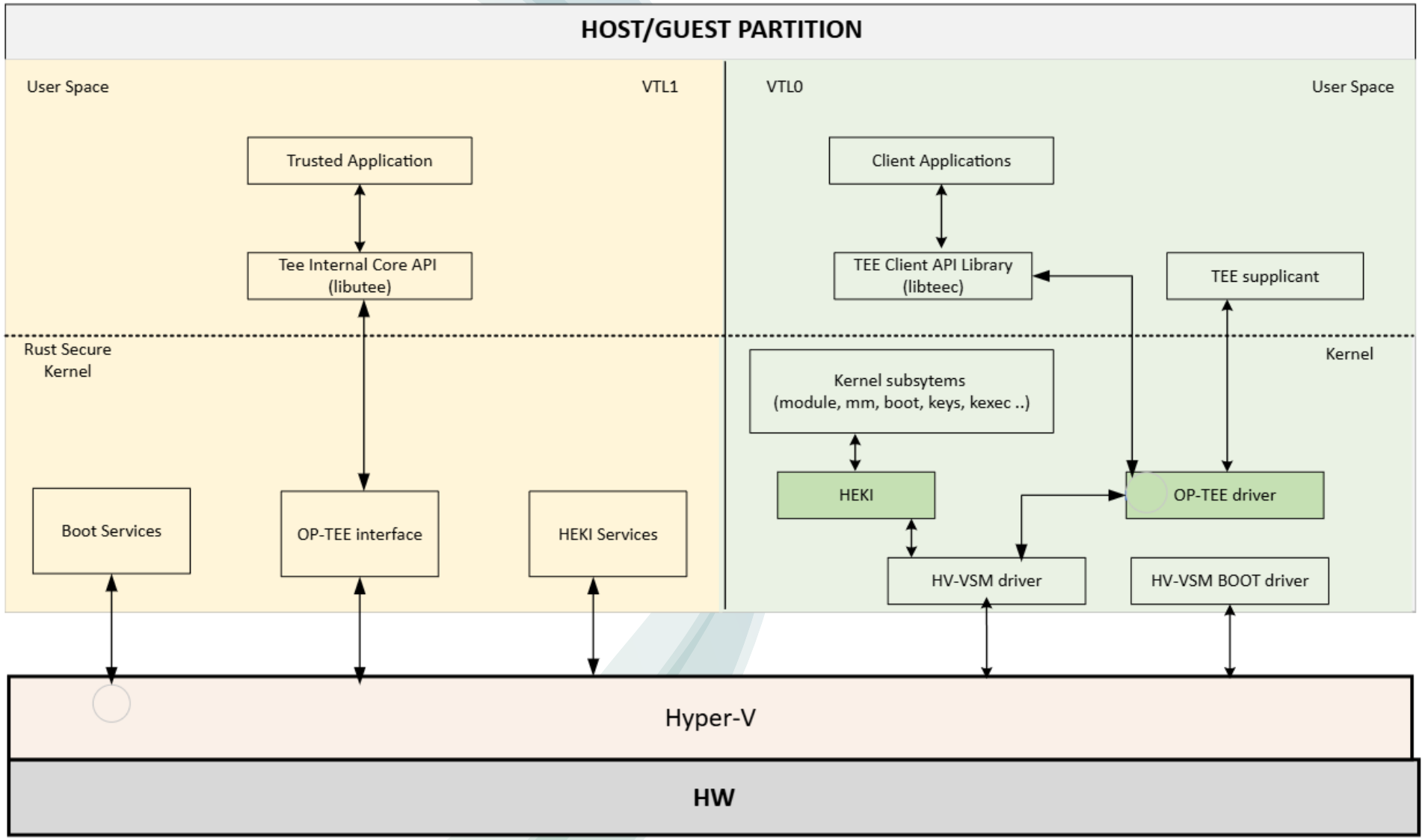
# Secure Kernel



- Small TCB and dependency
  - Memory Safey
- Ability to support secure interfaces
- Maintainability

- Evaluated Solutions
  - Minimal Linux Kernel
  - Optee Secure OS
  - Coco SVSM
  - **Rust Based minimal firmware**

# Putting it all together ....



# Where do we stand with KVM ?

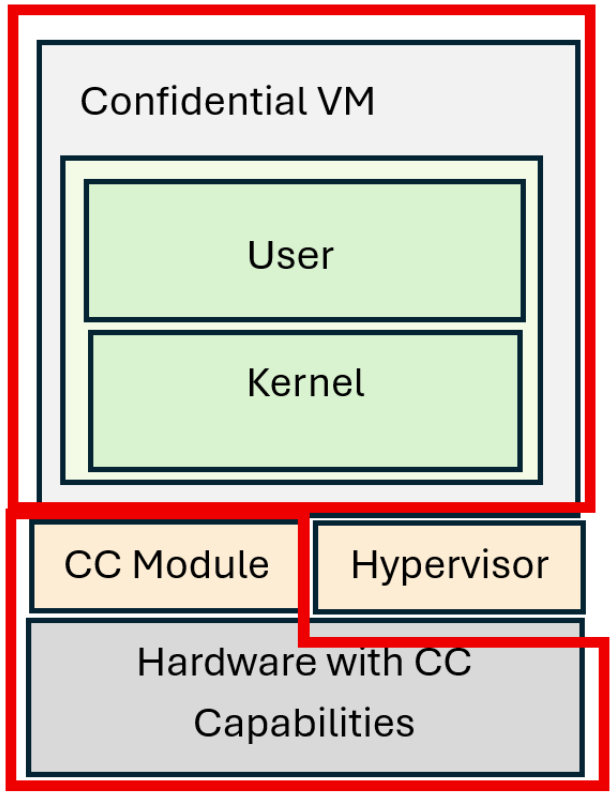
# KVM Status

- **Initial Efforts :**
  - Implement MBEC and register pinning in host KVM for guest VMs
  - RFC Patches: https://lore.kernel.org/all/20231113022326.24388-1-mic@digikod.net/
  - No creation of higher privilege environment
- **VM Planes :**
  - **https://lwn.net/Articles/1016113/**

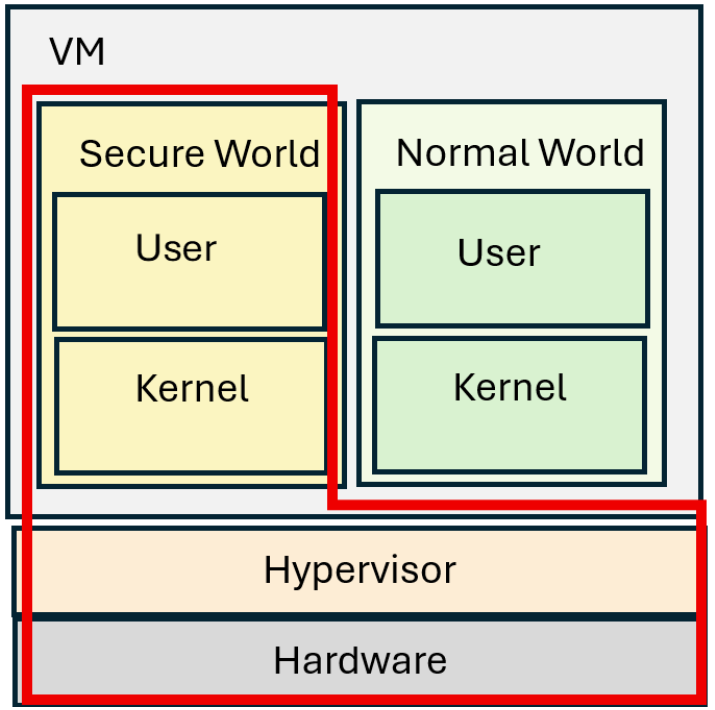
# Status

- Code : https://github.com/LinuxVBS/linux
- Please collaborate on:
  - Reviewing  Heki code
  - Reviewing VMPlanes and implementing VTLs, VMPLs etc with it
  - Optee x64 support

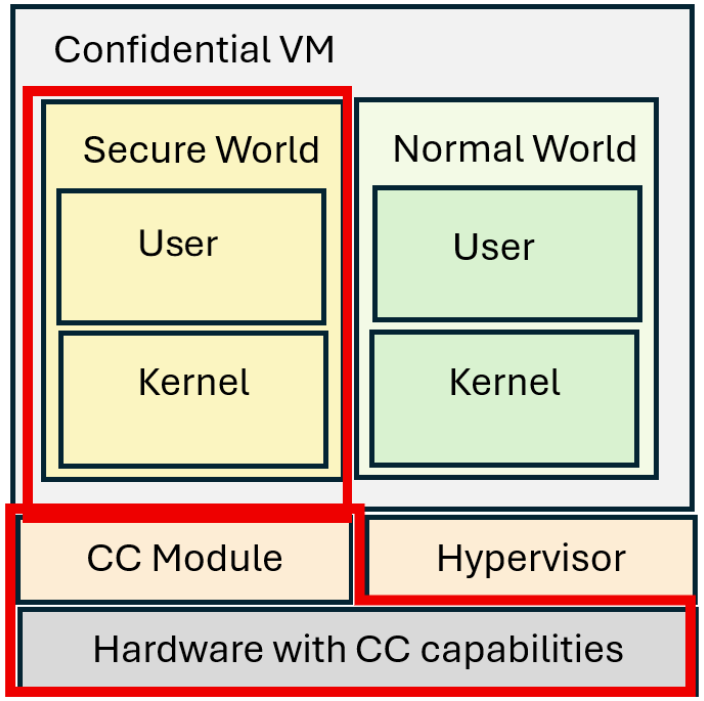
# Confidential Computing and LVBS




**TCB: Confidential VM**




**TCB : VM with LVBS**




**TCB : Confidential VM with LVBS**

# Q&A

